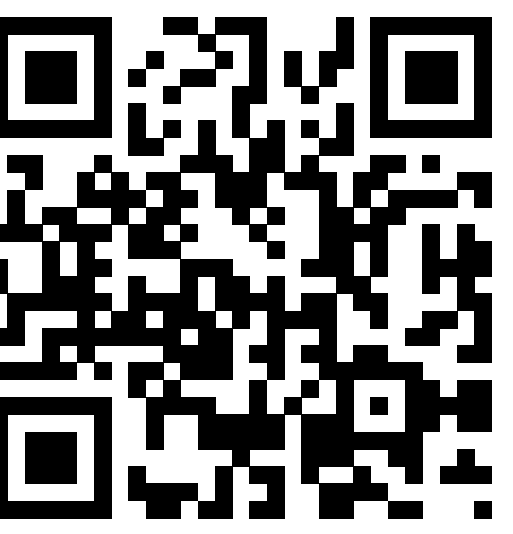


When Lifetimes Liberate

A Type System for Arenas with Higher-Order Reachability Tracking

Siyuan He¹ Songlin Jia¹ Yuyan Bao² Tiark Rumpf^f

¹Purdue University / PurPL ²Augusta University



Managing Resources in a Higher-Order Language

An **arena** groups resources and deallocates them in bulk.
This ends their **lifetime**, which follows one of two disciplines:

Lexical lifetime

```
{
  // one request
  val a = Arena()
  val hdr = a.alloc(...)
  val body = a.alloc(...)
  respond(hdr, body)
} // deallocated here
```

Non-lexical lifetime

```
val register = {
  val pool = Arena()
  (cb) => pool.alloc(cb)
} // pool escapes

register(onClick)
```

Second-class: deallocated at scope exit, so it cannot escape.

First-class: it may escape, but its reclamation is left to GC.

Our Goal: Unify the Strengths of Prior Systems

	Cyclone LIFO	Cyclone dynamic	Safe Rust ownership	Reggio region+OT	F _Δ reachability	This Work region+RT
Stack lifetime discipline	✓	✓	✓	✓		✓
Mutable, cyclic sharing	✓	✓		✓		✓
First-class escaping		✓	✓	✓	✓	✓
Higher-order functions	✓		✓		✓	✓

Grossman et al. PLDI'02 · Hicks et al. ISMM'04 · Arvidsson et al. OOPSLA'23 · Wei et al. POPL'24

One reference type: client code is generic over the storage model.

No use-after-free, guaranteed statically by flow-insensitive reasoning.

Background: Reachability Tracking

A type qualified by the set of resources the value may reach.

Binding: a fresh value ($\blacklozenge$) is tracked by its variable.

```
val r = new Ref(8) // new Ref(8): Ref[Int]♦
r //: Ref[Int]r
```

Aliases are tracked: x's qualifier upcasts to r.

```
val x = r //: Ref[Int]x <: Ref[Int]r
```

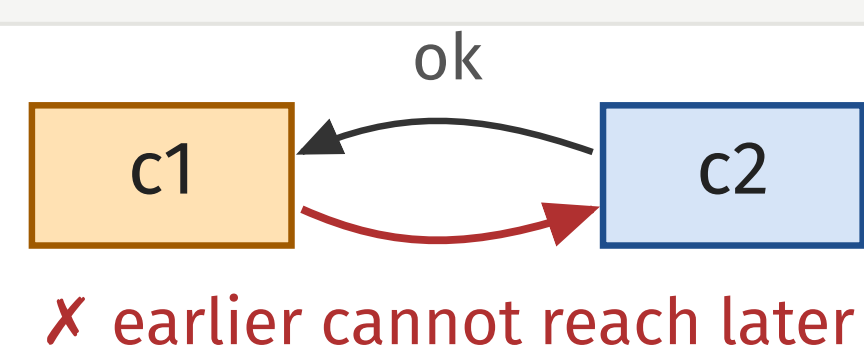
Closures track captured resources, even out of scope, via self-reference.

```
val f = () => !r //: (() => Int)r
val g = { val y = new Ref(9)
  () => y } //: (() => Ref[Int]g)g
```

Limitations of reachability types OOPSLA'21 · POPL'24 · OOPSLA'25

- No grouping:** qualifiers track individual locations only.
- No lifetime control:** reclamation is left to garbage collection.
- No cyclic store structures:** cells cannot reach later ones (telescoping).

```
val c1 = new Ref(f) //: Ref[(Int => Int)]q1c1
val c2 = new Ref(f) //: Ref[(Int => Int)]q2c2
c2 := x => (!c1)(x) // ok: q2 may mention c1
c1 := x => (!c2)(x) // error: q1 was fixed before c2
```



Referent qualifiers fixed at allocation rule out mutual recursion through the store.

Our Contribution: Arenas and Flexible Lifetimes

PRIOR	new Ref(t)	:	Ref[T] [♦]	fresh arena, non-lexical lifetime
NEW	new Ref(t) at r	:	Ref[T] ^r	same arena and lifetime as r
NEW	new Ref(t) scoped	:	Ref[T] [♦]	fresh arena, lexical lifetime

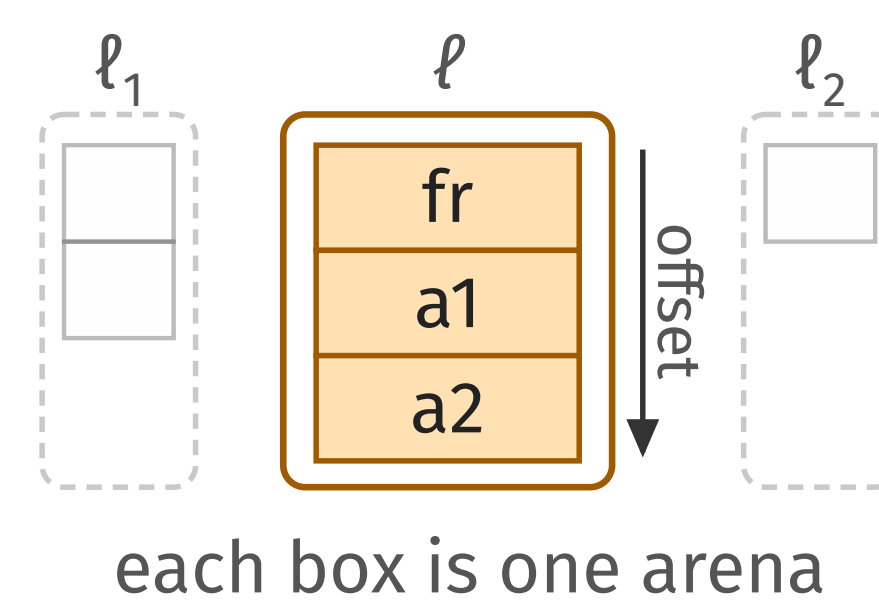
Coallocation (at) groups resources and permits cyclic store structures.

Scoped allocation (scoped) ties an arena's lifetime to its lexical scope.

One reference type for all three: they differ only in the qualifier, so code written for Ref[T] works with each of them.

Arenas: One Identity per Group

```
val fr = new Ref(42) //: Ref[Int]fr
val a1 = new Ref(7) at fr //: Ref[Int]a1 <: Ref[Int]fr
val a2 = new Ref(8) at a1 //: Ref[Int]a2 <: Ref[Int]fr
```



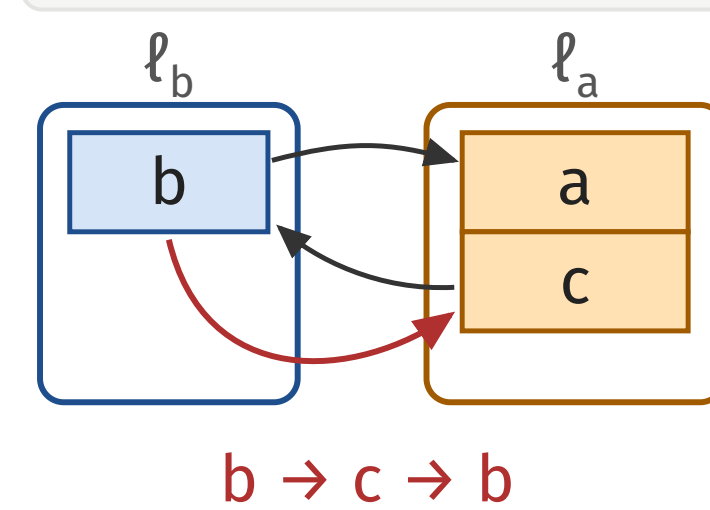
Shadow arenas have no explicit constructor:
new Ref creates one, **at** allocates into it.

Any member is a proxy: **at** fr and **at** a1 target the same arena.

Coarse-grained tracking: a cell is indexed by (ℓ , offset); qualifiers record only ℓ .

Arenas: Cycles Through the Store

```
val a = new Ref(..) //: Topa
val b = new Ref(a) //: Ref[Topa]b
val c = new Ref(b) at a //: Ref[Topb]c <: Topa
b := c // ok: closes b -> c -> b
```

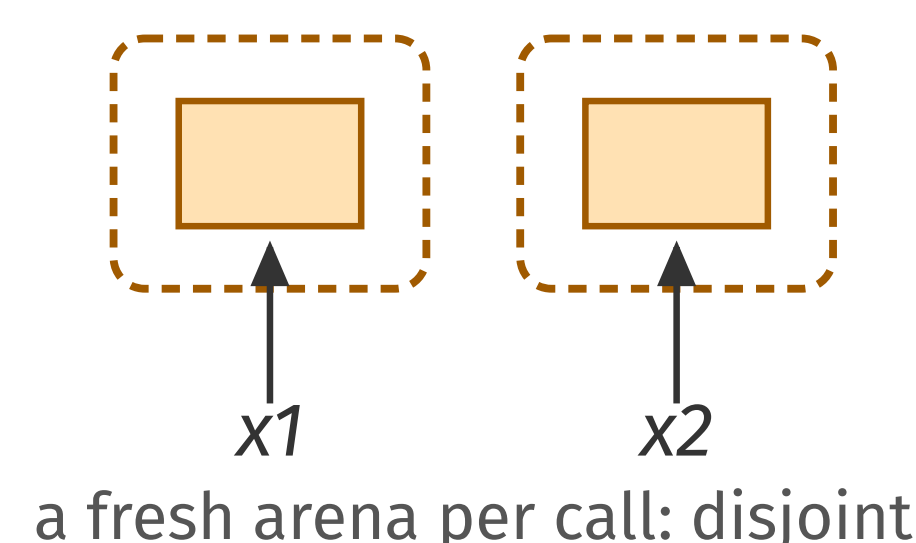


b := c requires only {a}, which b's referent already permits: c lies in a's arena.

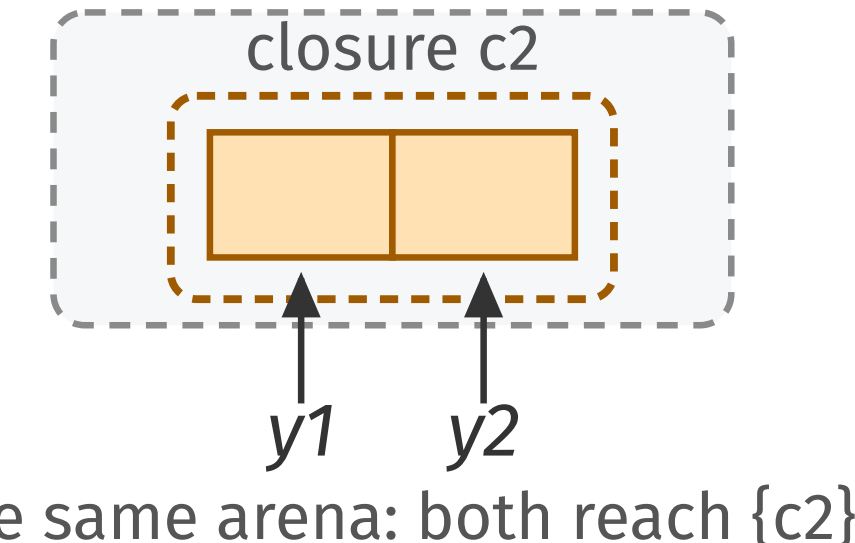
Telescoping is relaxed across arenas; cycles within one arena are the special case.

Non-Lexical Lifetimes: Arenas That Escape

```
def c1() = {
  val a = new Ref(())
  new Ref(7) at a
}
val x1 = c1() //: Ref[Int]x1
val x2 = c1() //: Ref[Int]x2
```



```
val c2 = {
  val b = new Ref(())
  () => new Ref(8) at b
}
val y1 = c2() //: Ref[Int]c2
val y2 = c2() //: Ref[Int]c2
```



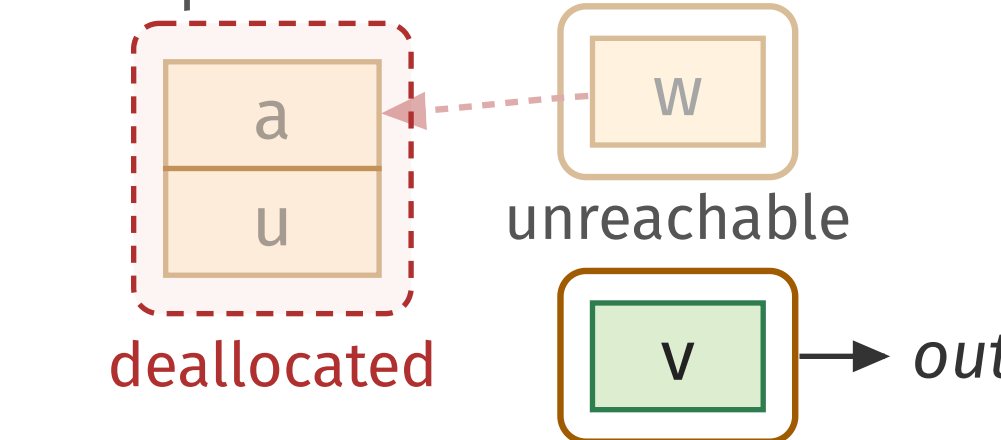
An escaping arena is tracked as an escaping reference.

Lexical Lifetimes: Why Deallocation Is Safe

scoped marks an arena for deallocation at scope exit. Within the scope it is used freely; after exit, any use must be ruled out.

```
val out = {
  val a = new Ref(()) scoped //: Ref[Unit]a
  val u = new Ref(42) at a //: Ref[Int]u <: Ref[Int]a
  val w = new Ref(a) //: Ref[Ref[Unit]a]w
  val v = new Ref(!u) //: Ref[Int]v
  v // ok: nothing in Ref[Int]v reaches a
} // a, u deallocated; w still refers to a
```

scoped arena



Returning u is rejected: {u} reaches {a}.

Storing a in w is allowed, but returning w is rejected: a occurs in its type.

w becomes unreachable at scope exit.

$$\frac{\Gamma^\varphi \vdash t_1 : T^p \quad \blacklozenge \notin p \quad (\Gamma, x : \text{Ref}[T^p]^\blacklozenge)^{\varphi, x} \vdash t_2 : Q \quad x \notin \text{fv}(Q)}{\Gamma^\varphi \vdash \text{ref } t_1 \text{ as } x \text{ in } t_2 : Q} \text{T-REFIN}$$

$x \notin \text{fv}(Q)$ makes the result deeply separate from the scoped arena.

Metatheory: No Use-After-Free Without Effects

At run time, locations such as ℓ appear in terms directly. The proof tracks the locations a term may **observe** (φ) and those already **deallocated** (κ).

	enter scope	$\rightarrow$	in scope	$\xrightarrow[t \Downarrow v']{} \rightarrow$	exit scope
term	ref v as x in t		local ℓ in $t[(\ell, 0)/x]$		v'
arena ℓ	unallocated		$(\ell, 0) \mapsto v$		deallocated
observable φ	φ		φ, ℓ		$\varphi \setminus \ell$
deallocated κ	κ		κ		κ, ℓ

$\varphi \cap \kappa = \emptyset$ is invariant: a well-typed term observes only φ , so it never reaches deallocated locations.

Removing ℓ from φ at close is sound: strengthening φ is not admissible in general, but a value is typable under its own qualifier, which **local** keeps free of ℓ .

$$\frac{\Gamma^\varphi \vdash t : T^q \quad \ell \notin q}{\Gamma^\varphi \vdash \text{local } \ell \text{ in } t : T^q} \text{T-LOCIN} \quad \frac{\Gamma^\varphi \vdash v : T^q \quad \ell \notin q}{\Gamma^\varphi \setminus \ell \vdash v : T^q} \text{VALUE RETYPING}$$

Sibling subterms never observe ℓ : in $(\text{local } \ell \text{ in } t_1) t_2$, the argument t_2 is typed under $\varphi \setminus \ell$, since ℓ is deallocated before t_2 is evaluated.

Progress and preservation hold for $A_\Delta^\blacklozenge$ (arenas; without **at**, it collapses to $F_\Delta^\blacklozenge$, POPL'24) and $\{A\}_\Delta^\blacklozenge$ (with scoped allocation), mechanized in Rocq.



Paper: doi.org/10.1145/3798254
Extended version: arxiv.org/abs/2509.04253
Rocq proofs: github.com/tiarkrumpf/reachability
Contact: he662@purdue.edu

Siyuan He is seeking academic and industry positions.